

Lead technique - TP Memory store

A l'heure actuelle, votre API fonctionne bien pour son but premier mais elle est susceptible d'être attaquée en DDoS. Le zippage de fichier est d'ailleurs encore plus sensible à un acteur malicieux spammant le bouton de génération de ZIP.

I. Introduction (prière de lire !)

Afin d'éviter ce genre de comportement, un concept de rate limitation est souvent mis en place dans les produits. Les utilisateurs seront limités dans leur nombre de requêtes en fonction de facteur les différenciant (adresse ip, compte connecté ou autre)

Une des manières simple et efficace de faire de la rate limitation est d'utiliser un token bucket algorithm. Cet algorithme a de multiples avantages, notamment:

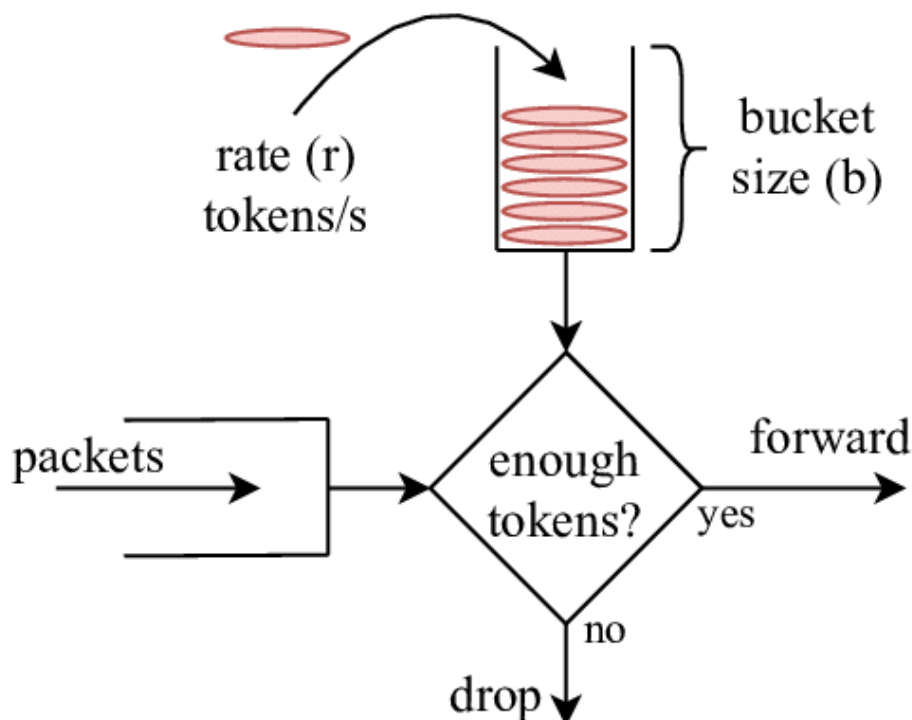
- Très peu coûteux en terme de stockage (2 nombre à stocker par rate limitation uniquement)
- Peut autoriser un burst de départ pour ensuite être plus restrictif en cas d'abus

Info:

Token -> Jeton en français

Bucket -> Seau en français

Pour vous mettre dans le bain, vous trouverez une explication théorique du token bucket algorithm ci-dessous.



Une requête peut avoir 2 issues possibles: une réussite (forward dans l'image) et un échec (drop dans l'image).

Trois données sont stockées en base de donnée:

- Un identifiant pour l'utilisateur
- La date du dernier comptage des jetons restants pour un utilisateur
- Le nombre de jeton restant pour l'utilisateur à cette date

En terme de configuration, il y a 3 nombres à mettre en constante dans votre code:

- Le nombre de jetons qu'un utilisateur va récupérer par seconde (r dans l'image)
- Le nombre de jetons avec lequel il démarre, qui est aussi son nombre de jetons max (b dans l'image)
- Le coût en jeton d'une requête (non représenté dans le schéma)

Exemple:

On définit un rate limiter avec les propriétés suivantes:

- Un utilisateur va récupérer un jeton par seconde (donc 60 par minutes)
- Le nombre de jetons de démarrage est 10
- Une requête va coûter un jeton.

Si un tout nouvel utilisateur (nommé U1) fait 3 requêtes par seconde, l'effet va être le suivant:

T-0 - Pas encore dans la base de données

T+1s -

Requête 1: On stocke dans la BDD le triplet (U1, T+1, 9)

Requête 2: On stocke dans la BDD le triplet (U1, T+1, 8)

Requête 3: On stocke dans la BDD le triplet (U1, T+1, 7)

T+2s -

Requête 3: On stocke dans la BDD le triplet (U1, T+1, 6)

Requête 4: On stocke dans la BDD le triplet (U1, T+1, 5)

Requête 5: On stocke dans la BDD le triplet (U1, T+1, 4)

T+3s -

Requête 6: On stocke dans la BDD le triplet (U1, T+1, 3)

Requête 7: On stocke dans la BDD le triplet (U1, T+1, 2)

Requête 8: On stocke dans la BDD le triplet (U1, T+1, 1)

T+4s -

Requête 9: On stocke dans la BDD le triplet (U1, T+1, 0)

Requête 10: Après vérification dans la BDD, on arriverait à un nombre négatif avec une soustraction du coût de la requête, on va donc réinitialiser son nombre de jetons avec le temps qui s'est écoulé depuis la précédente réinitialisation, donc T+1.

Entre T+1 et T+4, il y a eu 3 secondes, on ajoute donc ses tokens restants actuels (0) à $3 * \text{la vitesse de récupération de tokens (donc 1 par seconde)}$ ce qui nous fait $0 + 3 * 1 = 3$. On y soustrait le coût de la requête actuelle (1), ce qui nous fait $3 - 1 = 2$. On stocke alors dans la base de données (U1, T+4, 2)

Requête 11: On stocke dans la BDD le triplet (U1, T+4, 1)

T+5s -

Requête 12: On stocke dans la BDD le triplet (U1, T+4, 0)

Requête 13: Après vérification dans la BDD, on arriverait à un nombre négatif avec une soustraction du coût de la requête, on va donc réinitialiser son nombre de jetons avec le temps qui s'est écoulé depuis la précédente réinitialisation, donc **T+4**.

Entre **T+4 et T+5**, il y a eu 1 seconde, on ajoute donc ses tokens restants actuels (0)

à $1 * \text{la vitesse de récupération de tokens (donc 1 par seconde)}$ ce qui nous fait $0 + 1 * 1 = 1$. On y soustrait le coût de la requête actuelle (1), ce qui nous fait $1 - 1 = 0$. On stocke alors dans la base de données (U1, T+5, 0)

Requête 14: Après vérification dans la BDD, on arriverait à un nombre négatif, on va donc **TENTER** réinitialiser son nombre de jetons avec le temps qui s'est écoulé depuis la précédente réinitialisation. Entre T+5 et T+5, il n'y a pas eu suffisamment de jetons récupérés pour allouer la requête, elle est donc droppée. T+6s - La première requête passe avec une réinitialisation, on stocke (U1, T+6, 0) Les deux prochaines requêtes de la seconde sont refusées.

II. Implémentation de l'algorithme

Implémentez dans votre API un token bucket algorithm qui utilisera:

-Comme outils de différenciation l'ip de l'utilisateur (code ci-dessous pour récupérer l'ip en node)

```
var ip = req.headers['x-forwarded-for'] ||  
  req.socket.remoteAddress ||  
  null;
```

- 1 comme vitesse de récupération de jetons (r)
- 15 comme allocation de base (taille du seau, b)
- 3 comme coût en jeton pour une requête

Pour commencer simplement, le stockage de votre triplet (utilisateur, date, nombre de jetons) sera stocké dans une map avec pour clef l'ip de l'utilisateur dans votre app. Pas de persistance de données pour commencer.

III. Stockage de l'information dans une In-memory Database

En imaginant une scalabilité horizontale du produit, plusieurs instances seront mises côte à côte de la même API. La contrainte est que chacun aura sa propre rate limitation en l'état. Alors un acteur malveillant ne sera finalement pas si limité.

Pour parer à cette éventualité, il existe des bases de données en mémoire que l'on peut utiliser dans des architectures scalables horizontalement (type Redis).

Afin d'améliorer notre rate limitation, nous allons mettre ceci en place.

Utilisez la librairie [Redis](#) pour vous connecter à la BDD redis.

```
import { createClient } from 'redis';  
  
const client = createClient({  
  username: 'default',  
  password: 'vmIZrqNR0oso02oezzwaxBtfKcESh1Mw',  
  socket: {  
    host: 'redis-19512.cl6.us-east-1-2.ec2.cloud.redislabs.com',  
    port: 19512  
  }  
});
```



A nouveau, le mot de passe doit être mis dans une variable d'environnement et pas en clair dans le code. Il est ici juste pour vous transmettre celui que vous pourrez utiliser en développement. Veuillez ne pas le commit/push dans votre repository public.

Après vous être connecté à cette base grâce à la librairie, faites que votre tableau du token bucket soit stocké dans Redis.

Pour rappel, les In-memory databases (dont Redis) sont un système de stockage clef/valeur. (Ce qui correspond parfaitement à notre cas d'utilisation)